

Analiza Algoritmilor

P vs NP

Matei Popovici




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++]  := A[j++]
        else
                C[i++]  := B[k++]
    while (j < n)

C[i++]  := A[j++]
    while (k < m)
        C[i++]  := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

11110



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

00010



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    a := size of A
    n := size of B

C := new array of size a + n
    i,j,k := 0
    while (j < a and k < n)

if A[j] < B[k]

C[i++]  := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < n)
        C[i++] := B[k++]

    return C
```





```
function merge (A, B)
    a := size of A
    n := size of B

C := new array of size a + n
    i,j,k := 0
    while (j < a and k < n)

if A[j] < B[k]

C[i++]  := A[j++]
        else
                C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < n)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```


1 1 1 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 1 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 1 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 0 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 0 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 0 1 1
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 1 1 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

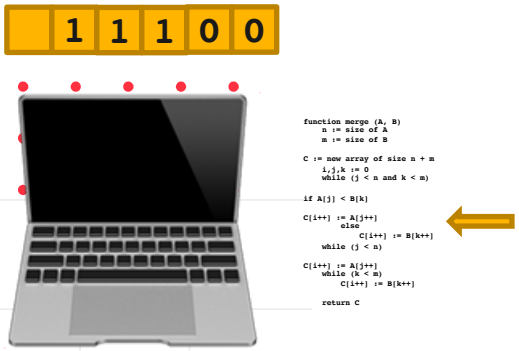

1 1 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

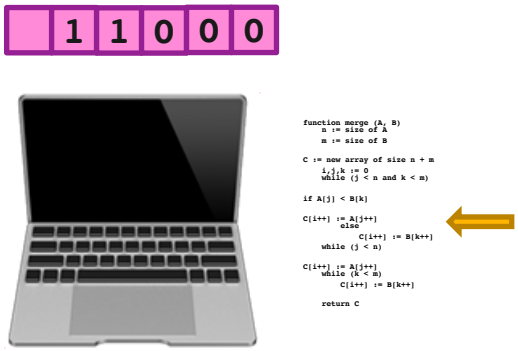

0 1 1 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

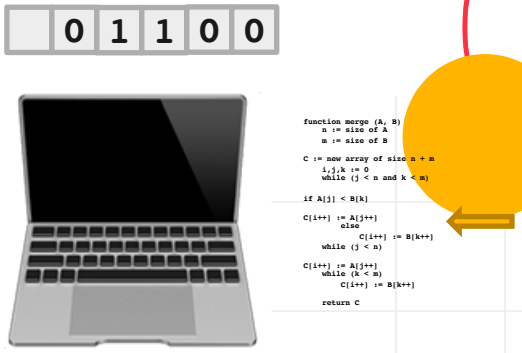

0 0 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

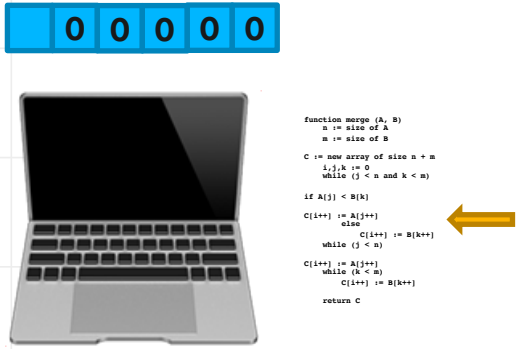

1 0 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

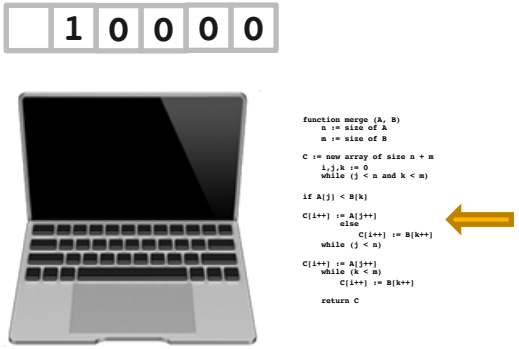

0 1 0 0 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

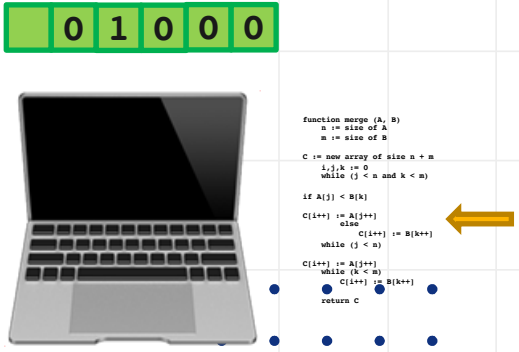

11100
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


11000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


01100
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


00000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


10000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


01000
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```




```
function merge (A, B)
    n := size of A
    m := size of B

C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)

if A[j] < B[k]

C[i++] := A[j++]
       else
            C[i++] := B[k++]
    while (j < n)

C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```


1 1 1 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 1 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 1 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 0 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


1 0 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C


0 1 0 1 0
function merge (A, B)
n := size of A
m := size of B
C := new array of size n + m
i,j,k := 0
while (j < n and k < m)
if A[j] < B[k]
C[i++] := A[j++]
else
C[i++] := B[k++]
while (j < n)
C[i++] := A[j++]
while (k < m)
C[i++] := B[k++]
return C

There is an unlimited number of copies the machine can make

Each machine works on its local data

Each machine has access to data computed before it was ***spawned***

Different copies cannot communicate or share data during their computations

Lecture outline

machine = algorithm

Formal definition

Can these machines solve undecidable problems?

Can these machines solve problems more efficiently?

Are there problems which still cannot be solved efficiently by these machines?

We will use these idealised machines to classify problems



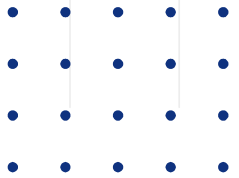
Lecture outline

We will use these idealised machines to classify problems

Problems which can be solved efficiently on non-parallel algorithms

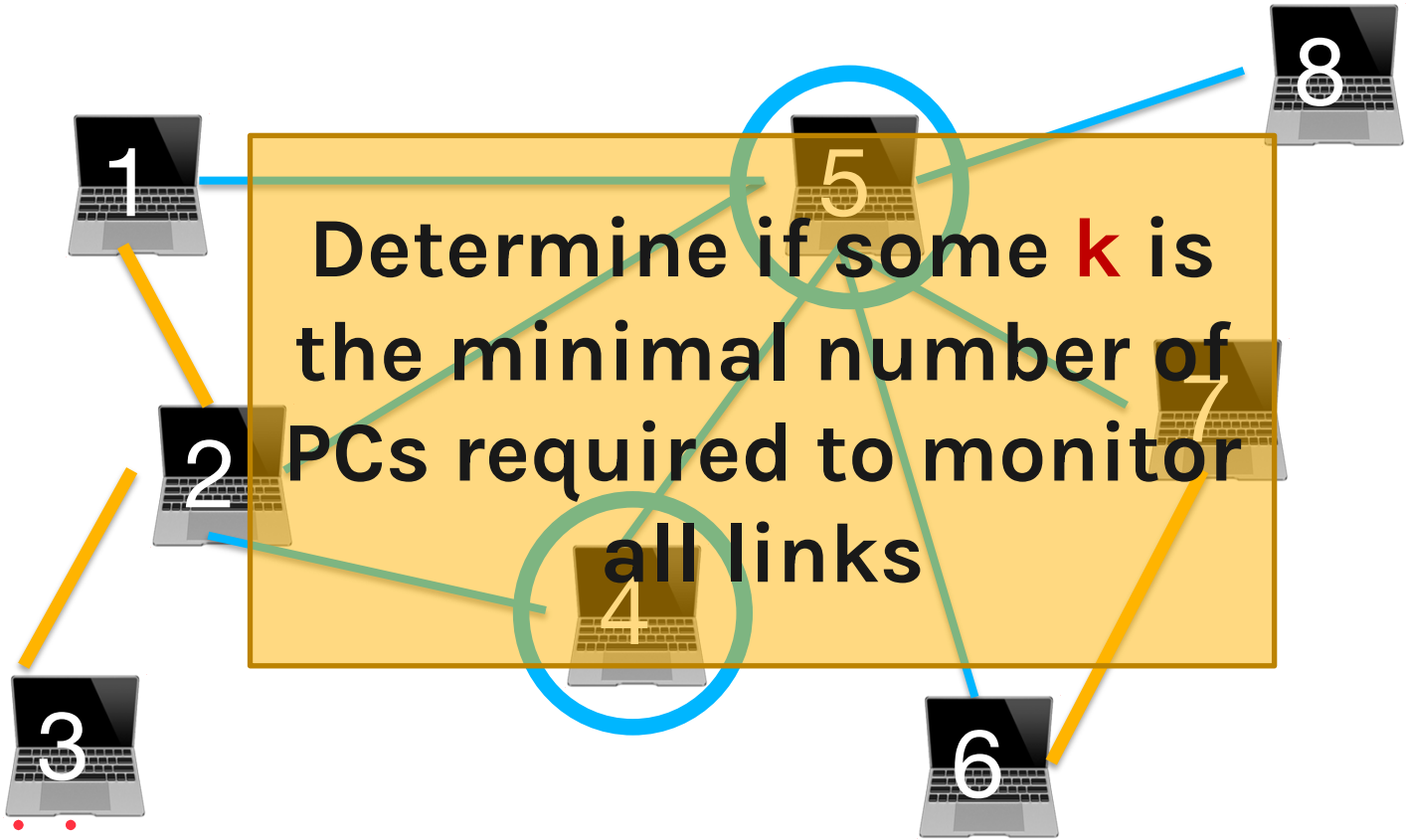
Problems which can be solved efficiently on parallel algorithms

Problems which cannot be solved efficiently on parallel algorithms



Parallel algorithms – informal definition

Problem 2: k-Vertex-Cover



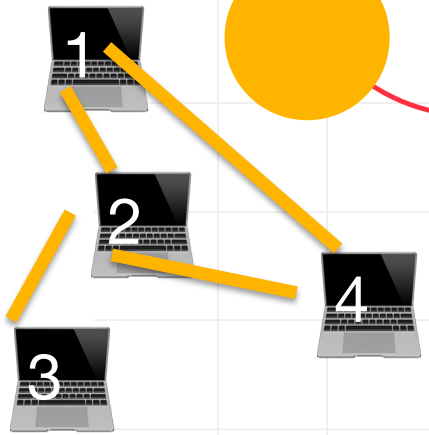
Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

$C = \emptyset,\ k = 2$



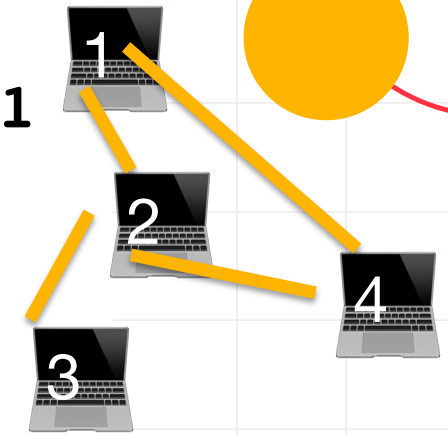
Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

$C = \emptyset,\ k = 1,\ v = 1$



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

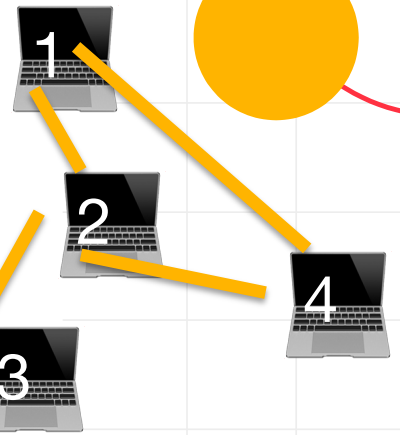
    success
```

c = ∅, k = 1, v = 1

c = ∅, k = 1, v = 2

c = ∅, k = 1, v = 3

c = ∅, k = 1, v = 4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

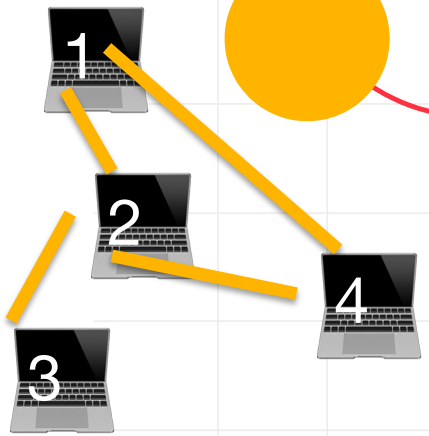
    success
```

C = {1}, k = 1

C = {2}, k = 1

C = {3}, k = 1

C = {4}, k = 1



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
  while (k > 0)
    k := k - 1
    v := choice(V)
    if C contains v
      fail
    C := {v} U C

  for each (u,v) in E
    if C does not contain u nor v
      fail

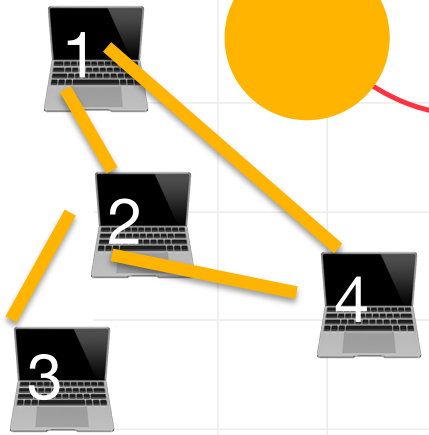
  success
```

C = {1}, k = 0

C = {2}, k = 0

C = {3}, k = 0

C = {4}, k = 0



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

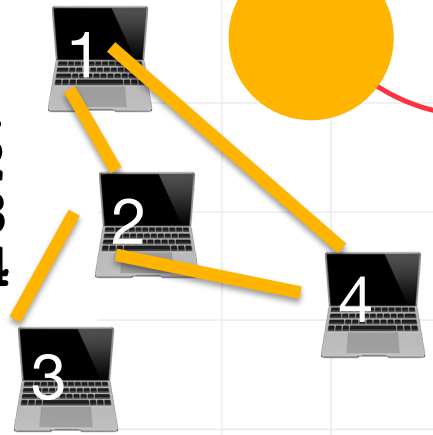
    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0

C = {3}, k = 0

C = {4}, k = 0



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

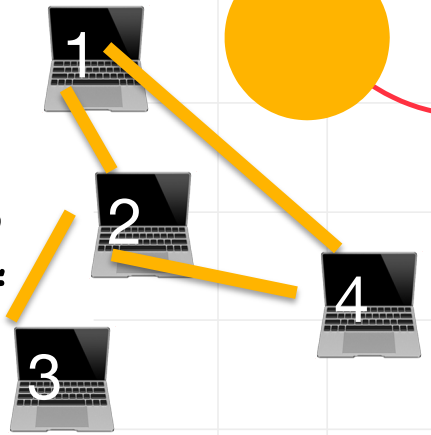
    success
```

```
C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0

C = {4}, k = 0
```



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

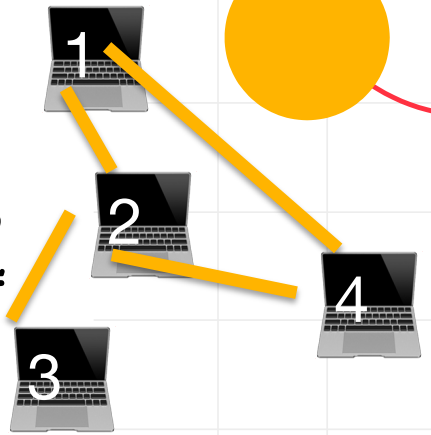
    success
```

```
C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0
```



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
        fail
      C := {v} U C

    for each (u,v) in E
      if C does not contain u nor v
        fail

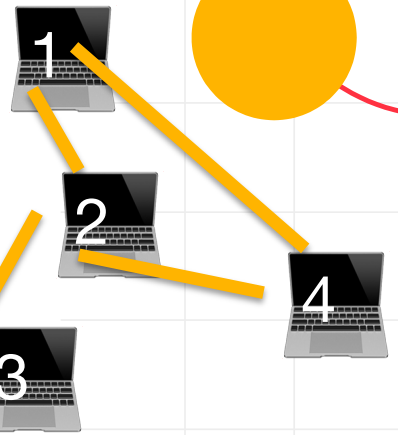
    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0, v=1
C = {4}, k = 0, v=2
C = {4}, k = 0, v=3
C = {4}, k = 0, v=4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} ∪ C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0, v=1
C = {4}, k = 0, v=2
C = {4}, k = 0, v=3
C = {4}, k = 0, v=4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} ∪ C

    for each (u,v) in E
       if C does not contain u nor v
          fail

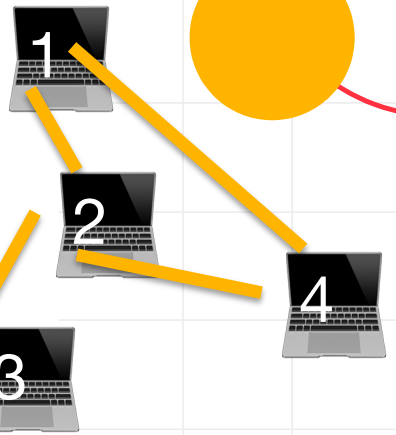
    success
```

C = {1}, k = 0, v=1
C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1
C = {2}, k = 0, v=2
C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2
C = {3}, k = 0, v=3
C = {3}, k = 0, v=4

C = {4}, k = 0, v=1
C = {4}, k = 0, v=2
C = {4}, k = 0, v=3
C = {4}, k = 0, v=4



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} ∪ C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {4}, k = 0, v=2
C = {4}, k = 0, v=3

C = {1}, k = 0, v=2
C = {1}, k = 0, v=3
C = {1}, k = 0, v=4

C = {2}, k = 0, v=1

C = {2}, k = 0, v=3
C = {2}, k = 0, v=4

C = {3}, k = 0, v=1
C = {3}, k = 0, v=2

C = {3}, k = 0, v=4

C = {4}, k = 0, v=1



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {1,2}
C = {1,3}
C = {1,4}

C = {2,1}

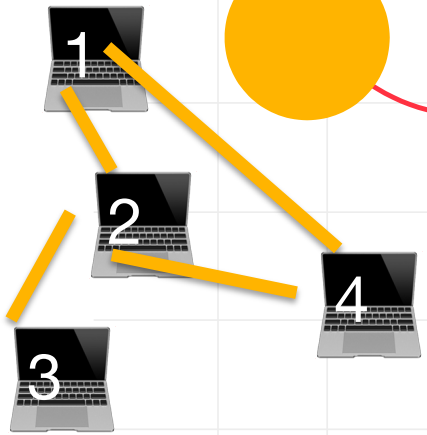
C = {2,3}
C = {2,4}

C = {3,1}
C = {3,2}

C = {3,4}

C = {4,1}

C = {4,2}
C = {4,3}



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

C = {1,2}
C = {1,3}
C = {1,4}

C = {2,1}

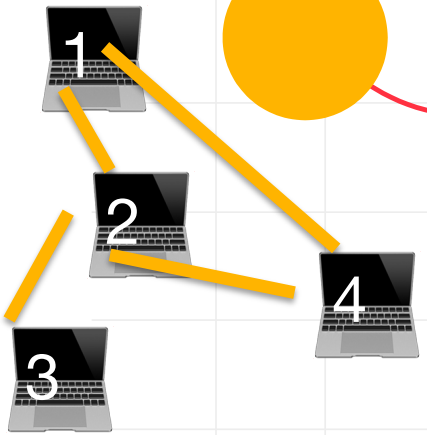
C = {2,3}
C = {2,4}

C = {3,1}
C = {3,2}

C = {3,4}

C = {4,1}

C = {4,2}
C = {4,3}



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

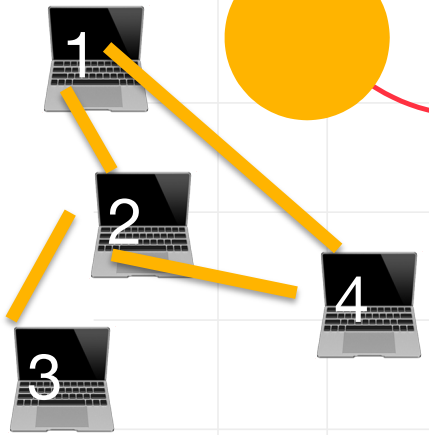
    success
```

C = {1,2}

C = {2,1}

C = {2,4}

C = {4,2}



Parallel algorithms – informal definition

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

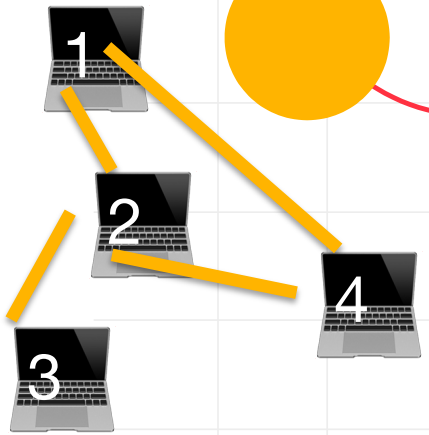
    success
```

C = {4,2}

C = {1,2}

C = {2,1}

C = {2,4}



Parallel algorithms – informal definition

Parallel algorithm = any type of pseudocode enhanced with:

`choice(A)`

Creates $|A|$ copies. For each value v_i in A, the algorithm spawns a clone. In each clone, this instruction evaluates to v_i.

`success`

Terminates the current copy with success.

`fail`

Terminates the current copy with failure.

Parallel algorithms – informal definition

A parallel algorithm algorithm returns true, iff at least one copy returns success.

A parallel algorithm algorithm returns false, iff all copies fail.

Parallel algorithms – informal definition

A parallel algorithm is nondeterministic, in the sense that its execution is not uniquely determined by the input data.

Parallel algorithms – complexity

nondeterministic complexity

angelic complexity

```
function k-Vertex-Cover(k,V,E)
    C := emptyset
    while (k > 0)
      k := k - 1
      v := choice(V)
      if C contains v
         fail
      C := {v} U C

    for each (u,v) in E
       if C does not contain u nor v
          fail

    success
```

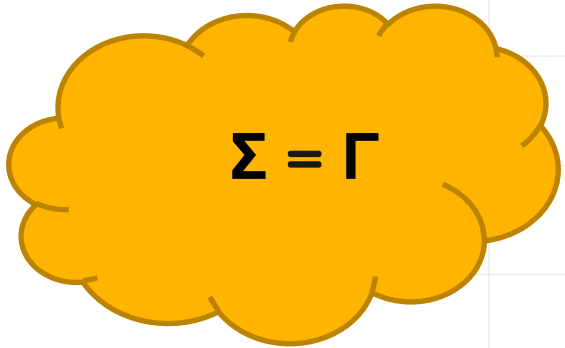
`choice(A)` constant time $O(1)$

Angelic complexity for this algorithm is $O(n^2)$

Deterministic Turing Machines

$$M = (Q, \Sigma, \Gamma, B, q_1, Y, N, H, \delta).$$

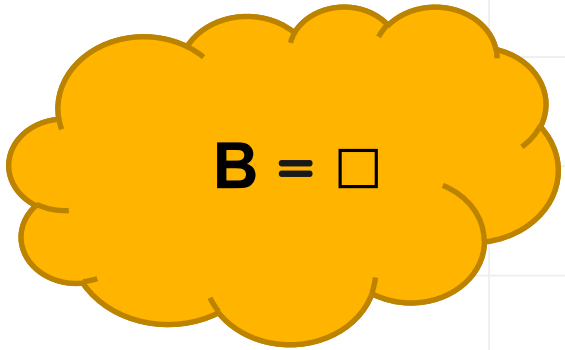
- Q – set of states
- Σ – input alphabet
- Γ – tape alphabet
- B – blank symbol
- q_1 – initial state
- Y, N, H – accepting, rejecting, halting states
- δ – transition function



Deterministic Turing Machines

$$M = (Q, \Sigma, B, q_1, Y, N, H, \delta).$$

- Q – set of states
- Σ – input alphabet
- B – blank symbol
- q_1 – initial state
- Y, N, H – accepting, rejecting, halting states
- δ – transition function



Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, H, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N, H – accepting, rejecting, halting states

δ – transition function

B = □

Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, H, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N, H – accepting, rejecting, halting states

δ – transition function

We do not need a halting state for decision problems

Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N – accepting, rejecting states

δ – transition function

Deterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

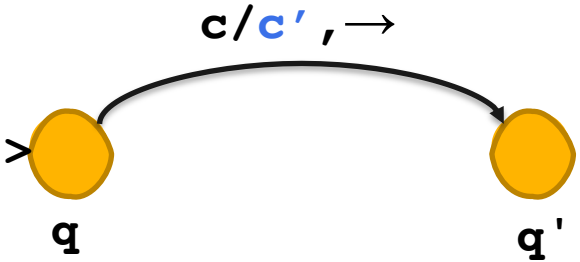
Q – set of states

Σ – input alphabet

q_1 – initial state

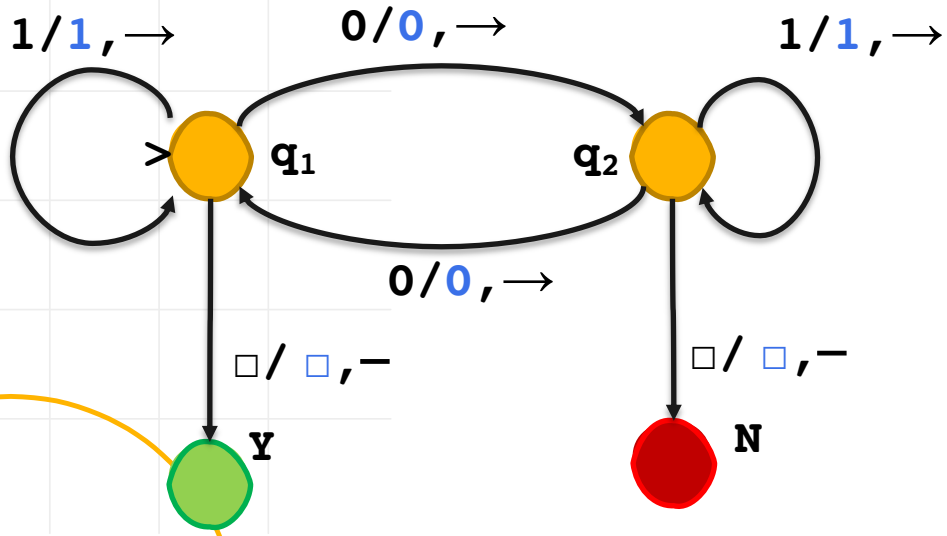
Y, N – accepting, rejecting states

$\delta: Q \times \Sigma \longrightarrow Q \cup \{Y, N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ – transition function



Deterministic Turing Machines

$M = (Q, \Sigma, q_1, Y, N, \delta)$.



	0	1	□
q_1	$q_2, 0, \rightarrow$	$q_1, 1, \rightarrow$	$Y, \square, -$
q_2	$q_1, 0, \rightarrow$	$q_2, 1, \rightarrow$	$N, \square, -$

Nondeterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

Y, N – accepting, rejecting states

$\delta: Q \times \Sigma \longrightarrow Q \cup \{Y, N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ – transition function

Nondeterministic Turing Machines

$$M = (Q, \Sigma, q_1, Y, N, \delta).$$

Q – set of states

Σ – input alphabet

q_1 – initial state

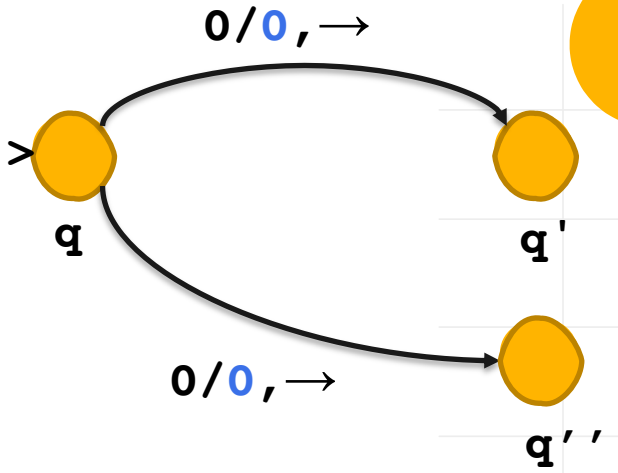
Y, N – accepting, rejecting states

~~$\delta: Q \times \Sigma \Longrightarrow Q \cup \{Y,N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$~~ ~~transition function~~

$\Delta: Q \times \Sigma \times Q \cup \{Y,N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ transition relation

Nondeterministic Turing Machines

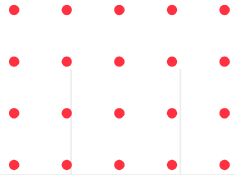
$M = (Q, \Sigma, q_1, Y, N, \Delta)$.

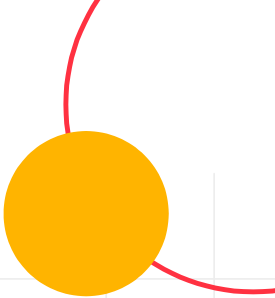


- Q – set of states
- Σ – input alphabet
- q_1 – initial state
- Y, N – accepting, rejecting states

$\Delta: Q \times \Sigma \times Q \cup \{Y, N\} \times \Sigma \times \{\leftarrow, -, \rightarrow\}$ – transition relation

The Turing Machine can now be in two or more states ***at the same time.***

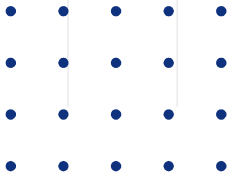




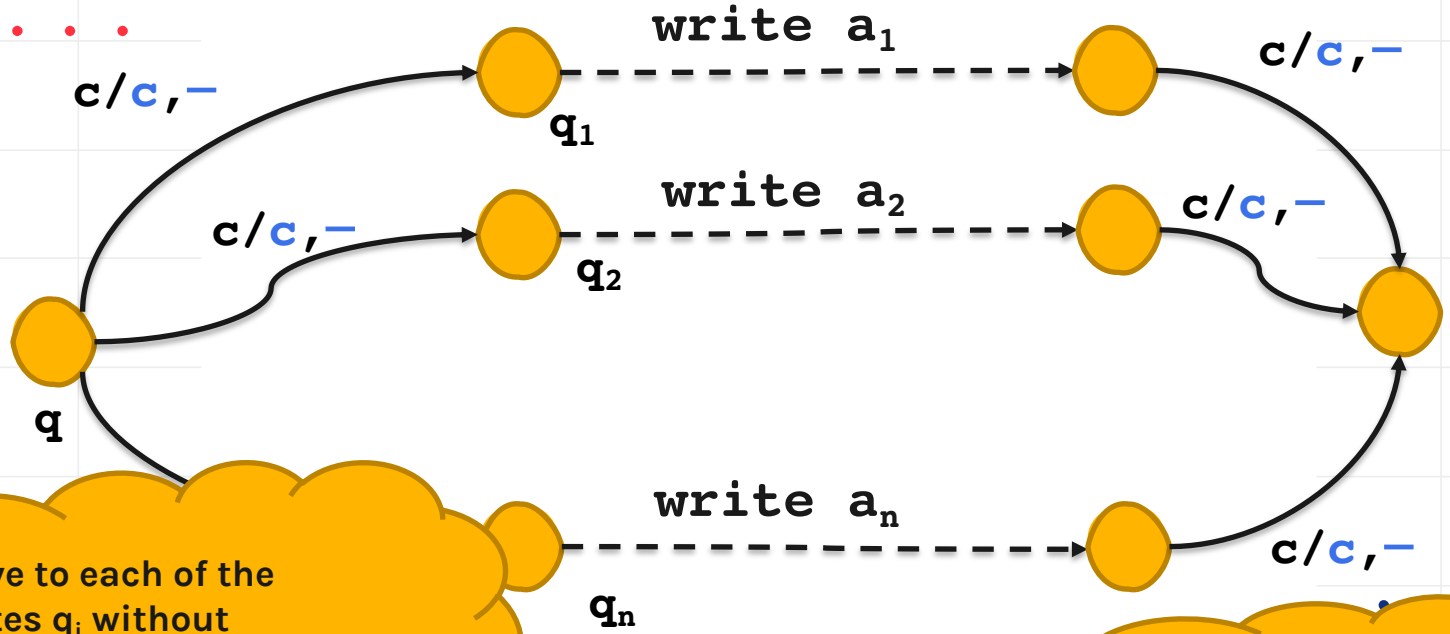
The relationship between ***Nondeterministic Turing Machines*** and ***Parallel Algorithms***

They are one and the same!



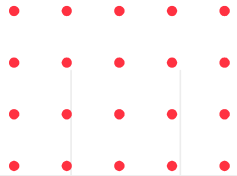


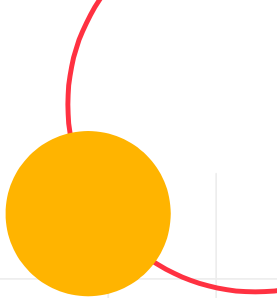
choice(A) where A = {a_1, a_2, …, a_n}



Move to each of the states q_i without changing the contents of the tape

Continue with the processing....

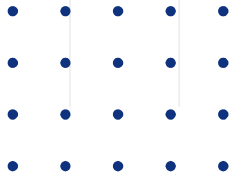




The relationship between ***Nondeterministic Turing Machines*** and ***Parallel Algorithms***

We will prefer writing parallel algorithms because they are more legible and easy to follow

Any parallel algorithm is a Nondeterministic Turing Machine

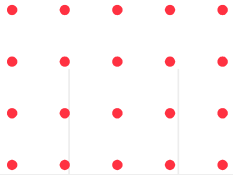




Complexity Theory – an introduction

Classify problems (not algorithms!) based on the existence of efficient algorithms for them.

If a problem can be solved by some algorithm in polynomial time, then we do not care about the exponent of the polynomial



polynomial

worse than polynomial

Turing Machines

graph reachability

is prime

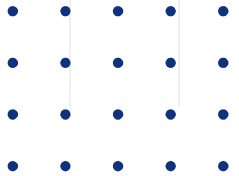
sorted sequence

Nondeterministic Turing Machines

graph reachability

is prime

sorted sequence



Turing Machines	Nondeterministic Turing Machines
polynomial	
worse than polynomial	k-vertex-covering
	hamiltonian path
	SAT
	All-k-vertex-covering
	forall-SAT

Turing Machines	Nondeterministic Turing Machines
polynomial	
worse than polynomial	k-vertex-covering
	hamiltonian path
	SAT
	All-k-vertex-covering
	forall-SAT

Turing Machines

Nondeterministic Turing Machines

It is expensive to go over possible candidate solutions

It is cheap to check if a given candidate is good

polynomial

k-vertex-covering

hamiltonian path

SAT

worse than polynomial

k-vertex-covering

hamiltonian path

SAT

***All*-k-vertex-covering**

***forall*-SAT**

Turing Machines

Nondeterministic Turing Machines

It is expensive to go over possible candidate solutions

It is cheap to check if a given candidate is good

polynomial

k-vertex-covering

hamiltonian path

SAT

worse than polynomial

k-vertex-covering

hamiltonian path

SAT

***All*-k-vertex-covering**

***forall*-SAT**



Turing Machines

Nondeterministic Turing Machines

It is **expensive** to go over possible candidate solutions

It is **cheap** to check if a given candidate is good

polynomial

k-vertex-covering

hamiltonian path

SAT

worse than polynomial

k-vertex-covering

hamiltonian path

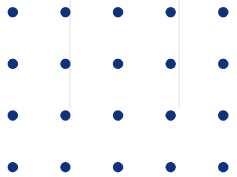
SAT

***All*-k-vertex-covering**

***forall*-SAT**

***All*-k-vertex-covering**

***forall*-SAT**

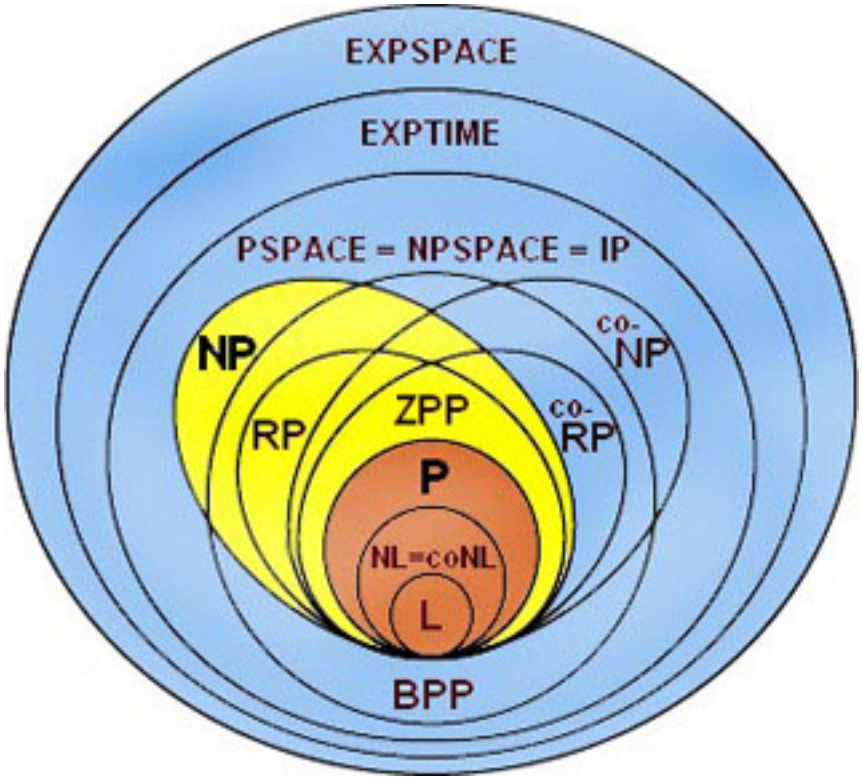


Complexity Theory – classifies problems into:

Problems that can be solved in deterministic polynomial time

Problems that can be solved in NON-deterministic polynomial time

Problems that can be solved in deterministic exponential time


EXPSPACE
EXPTIME
PSPACE = NPSPACE = IP
NP
co-NP
RP
ZPP
co-RP
P
NL=coNL
L
BPP

Complexity Theory – classifies problems into:

Problems that can be solved in deterministic poly

Problems tha NON-determin

Problems that can b deterministic exponential time

What happens when we need better algorithms for our problem at hand?

What happens when we need better algorithms?

Problems that can be solved in deterministic polynomial time

improve data-structures
run in data-center

Problems that can be solved in NON-deterministic polynomial time

Use SAT solvers
Use approximation algorithms
Use random algorithms

Problems that can be solved in deterministic exponential time

Use SMT solvers
Identify tractable cases

Complexity classes

Problems that can be solved in deterministic polynomial time

Problems that can be solved in NON-deterministic polynomial time

Problems that can be solved in deterministic exponential time

P = The set of all **decision problems** f for which there exists a Turing Machine/ algorithm that runs in **polynomial time** and **decides** f.

NP = The set of all **decision problems** f for which there exists a **Nondeterministic** Turing Machine/ algorithm that runs in **polynomial time** and **decides** f.

EXP = The set of all **decision problems** f for which there exists a Turing Machine/ algorithm that runs in **exponential time** and **decides** f.

Relationship between classes

$$P \subseteq NP \subseteq EXP$$

A deterministic Turing Machine is a special case of a nondeterministic Turing Machine

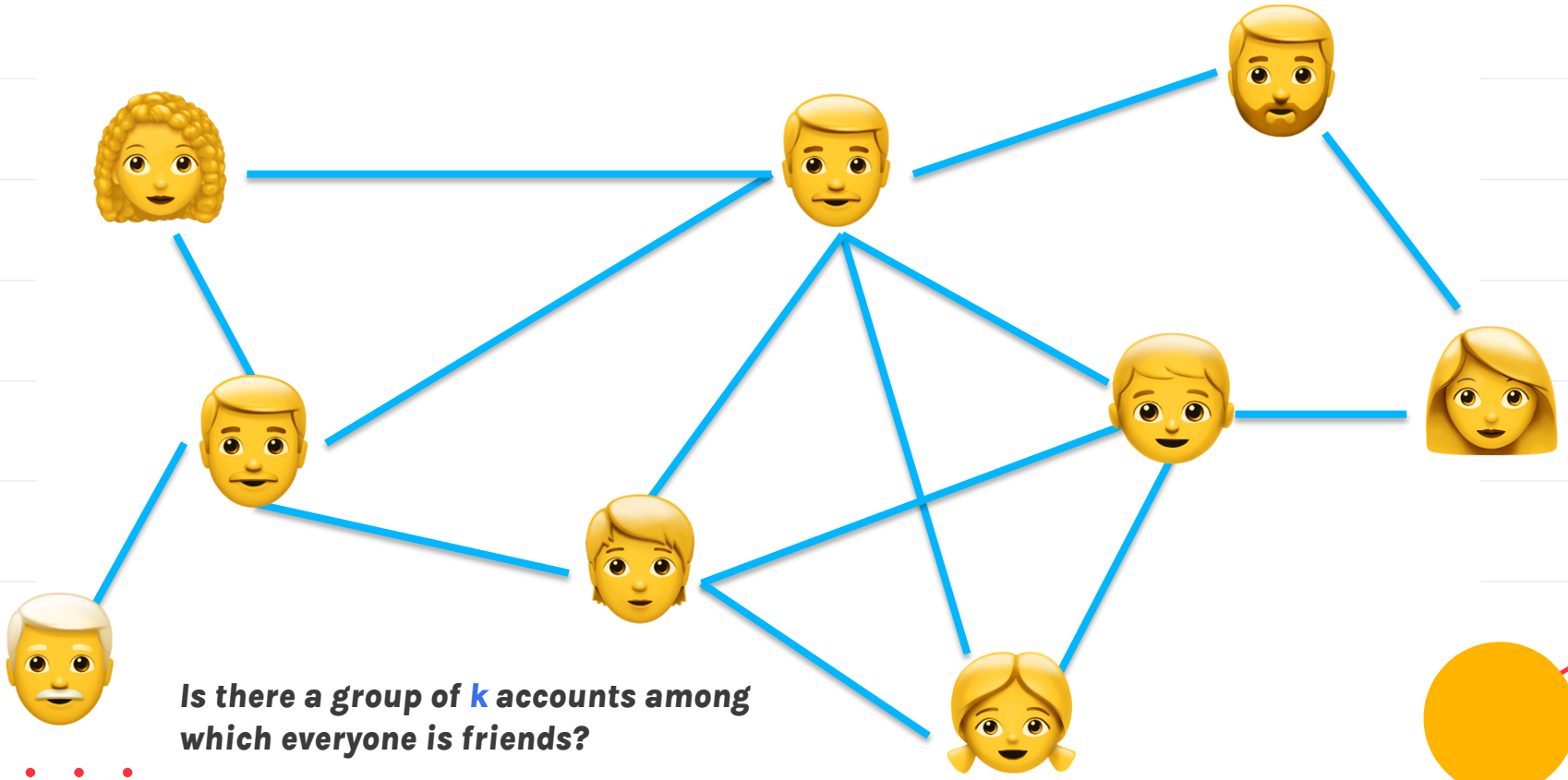
If a problem is solvable in deterministic polynomial time, it is also solvable in nondeterministic polynomial time (i.e. by a NTM)

Relationship between classes

$$P \subseteq NP \subseteq EXP$$

We can simulate any nondeterministic Turing Machine running in polynomial time or a parallel algorithm via backtracking, as a conventional Turing Machine/conventional algorithm that runs in exponential time.

Practice: social network

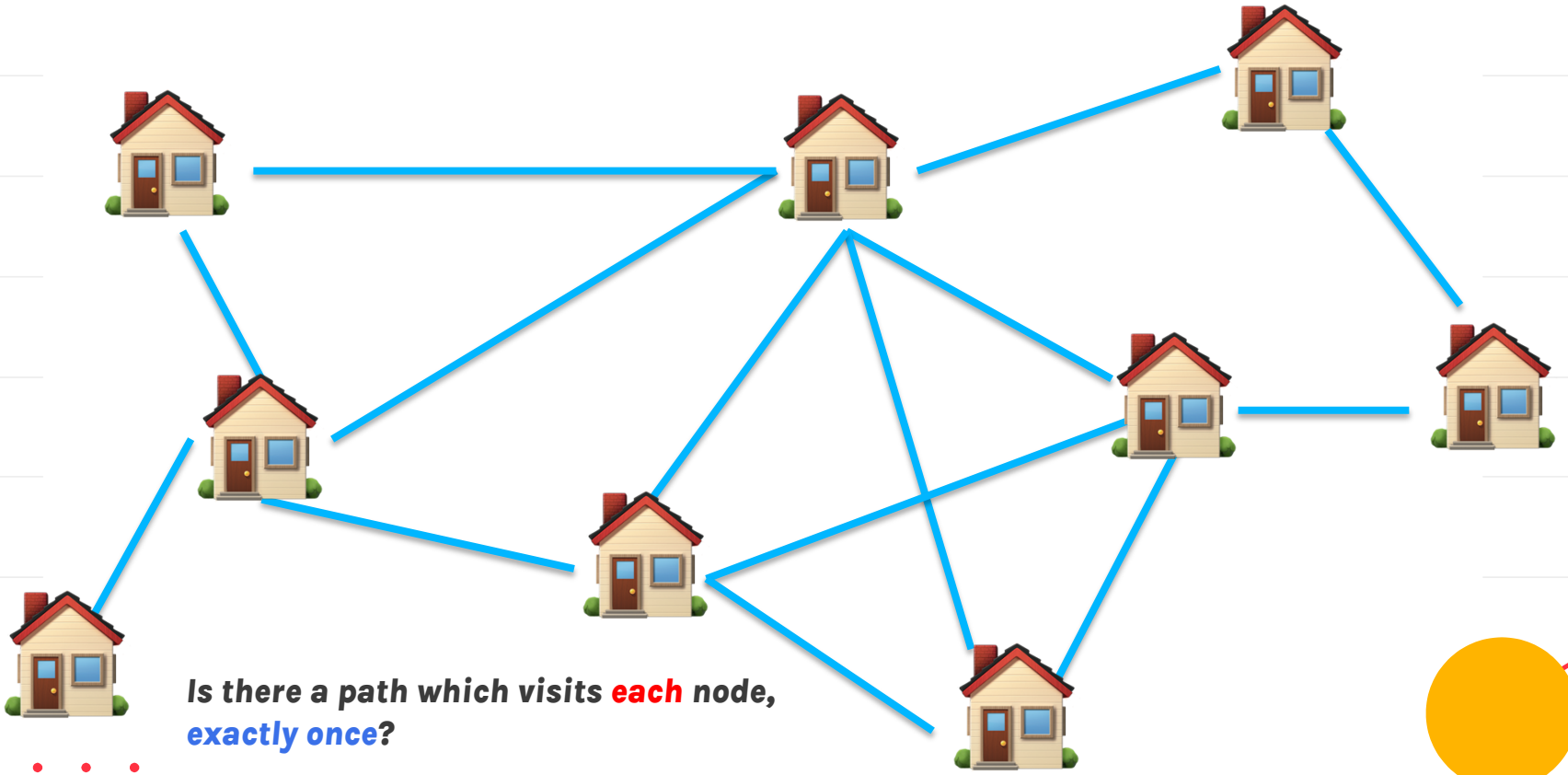


Is there a group of k accounts among which everyone is friends?

Practice: k clique

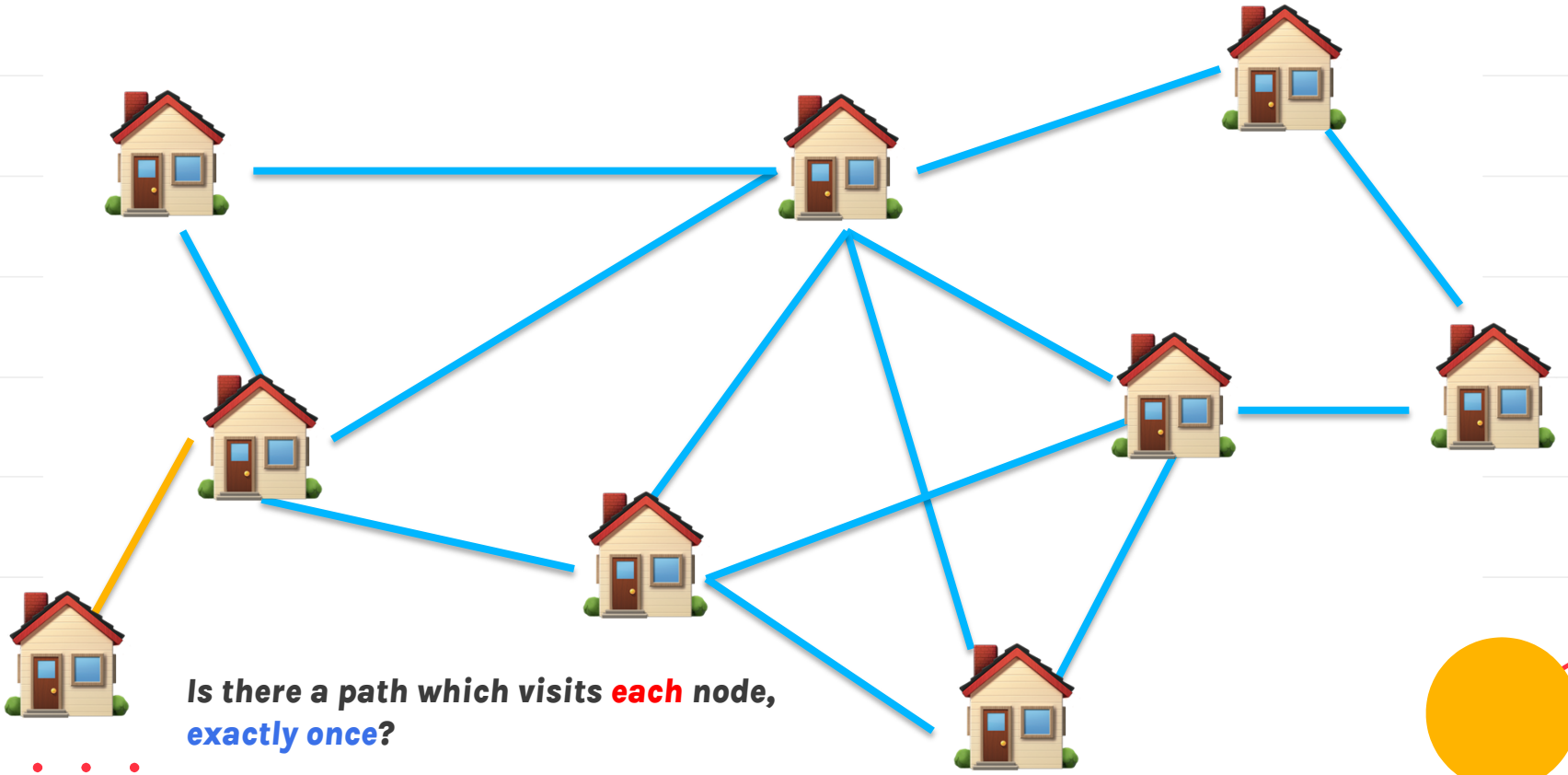
Let $G = (V,E)$ *be an unoriented graph and* k *be a natural number.*
There exists a subset $C \subseteq V$ *of size* k *such that, for each pair of nodes* $u, v \in C$, *there is an edge* $(u,v) \in E$?

Practice: Hamiltonian Path



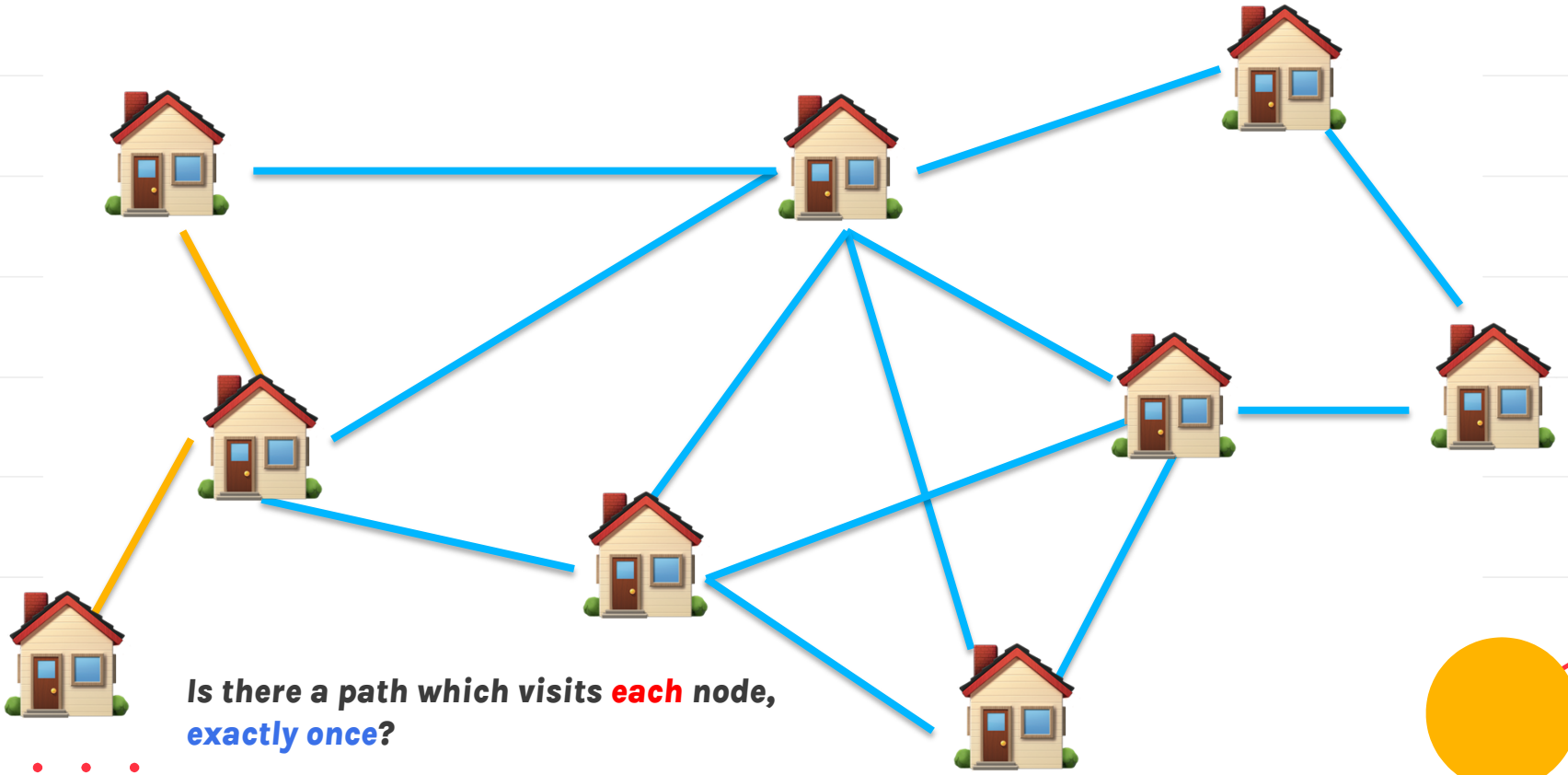
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



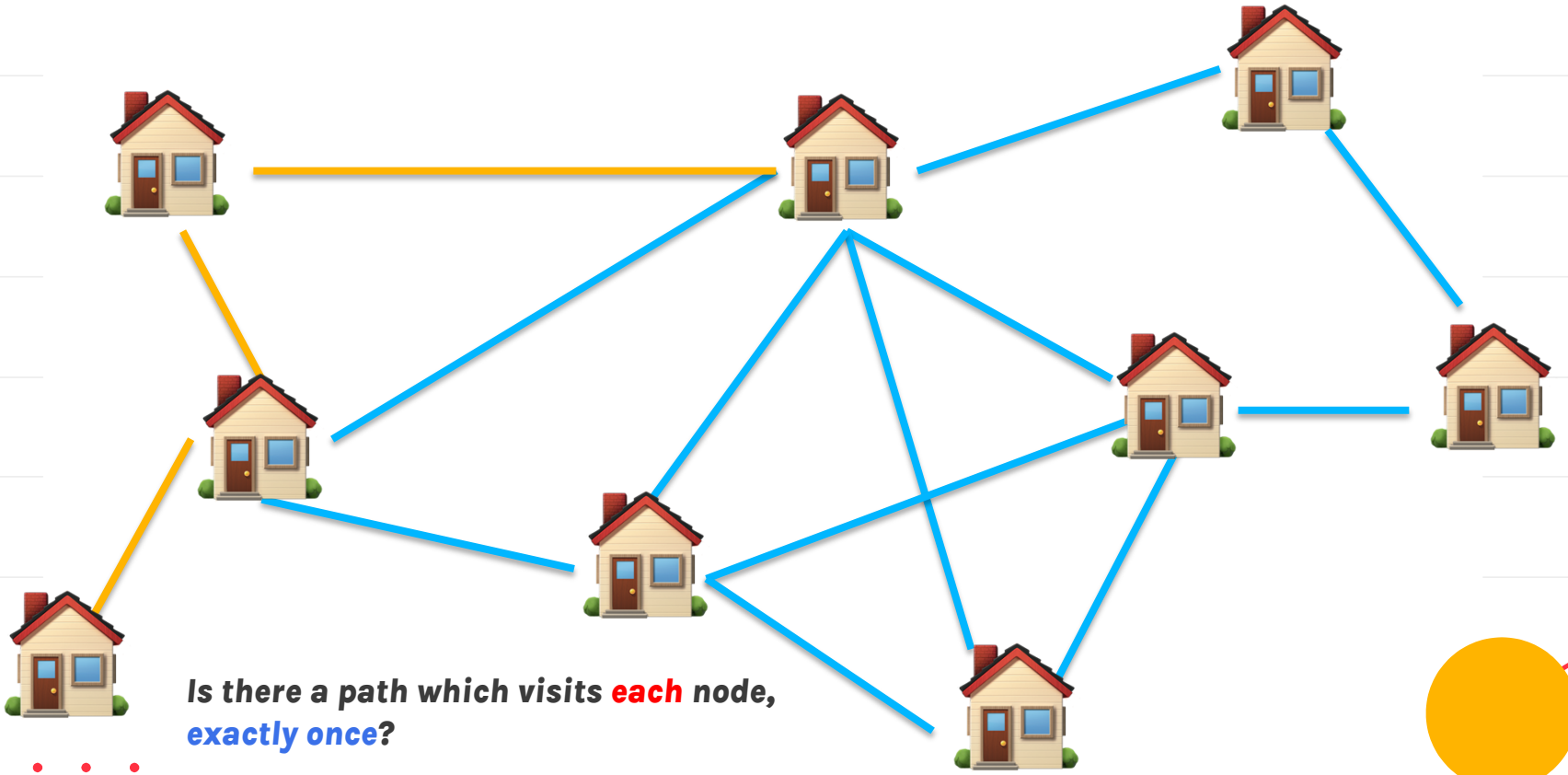
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



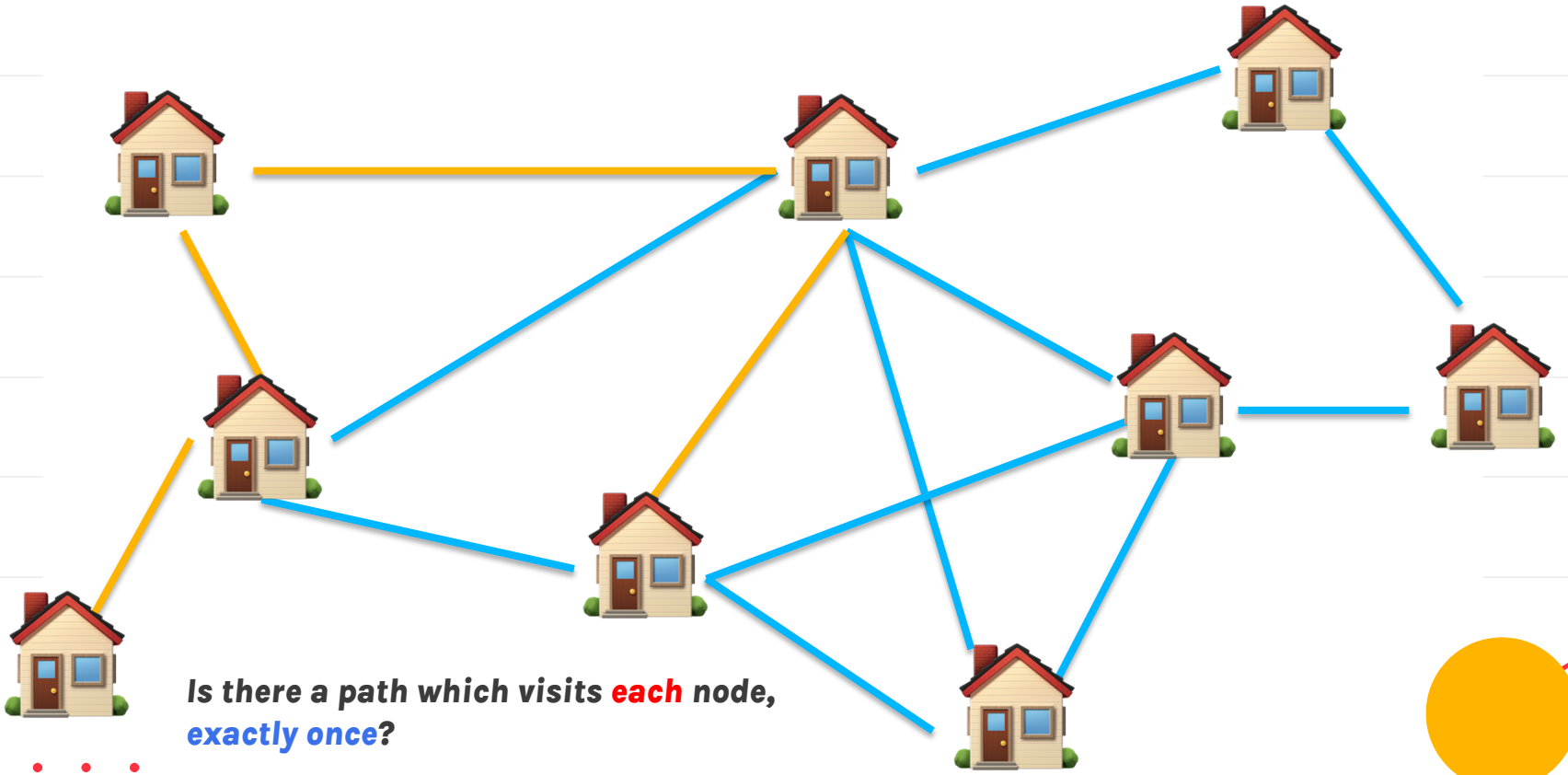
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



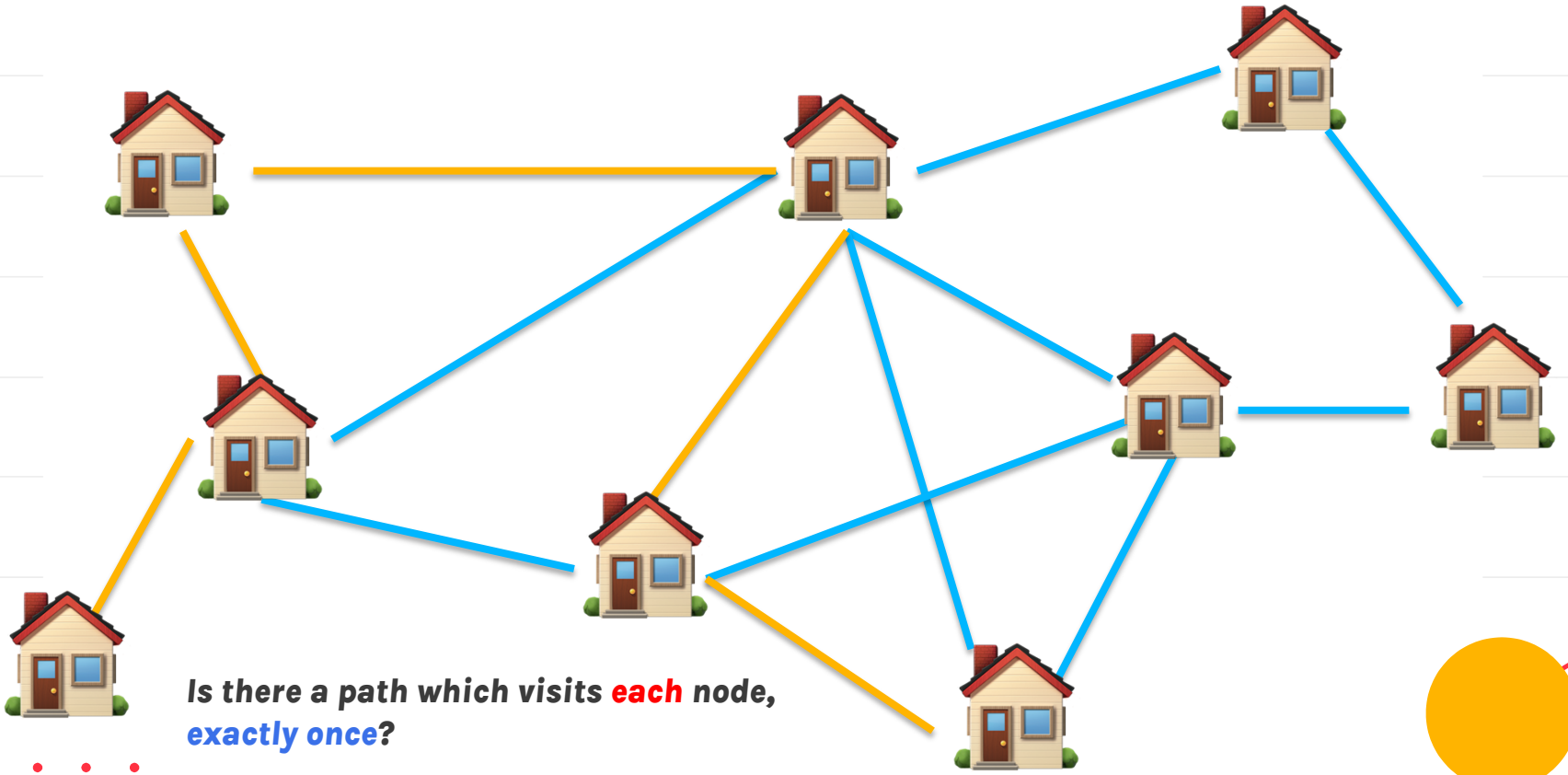
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



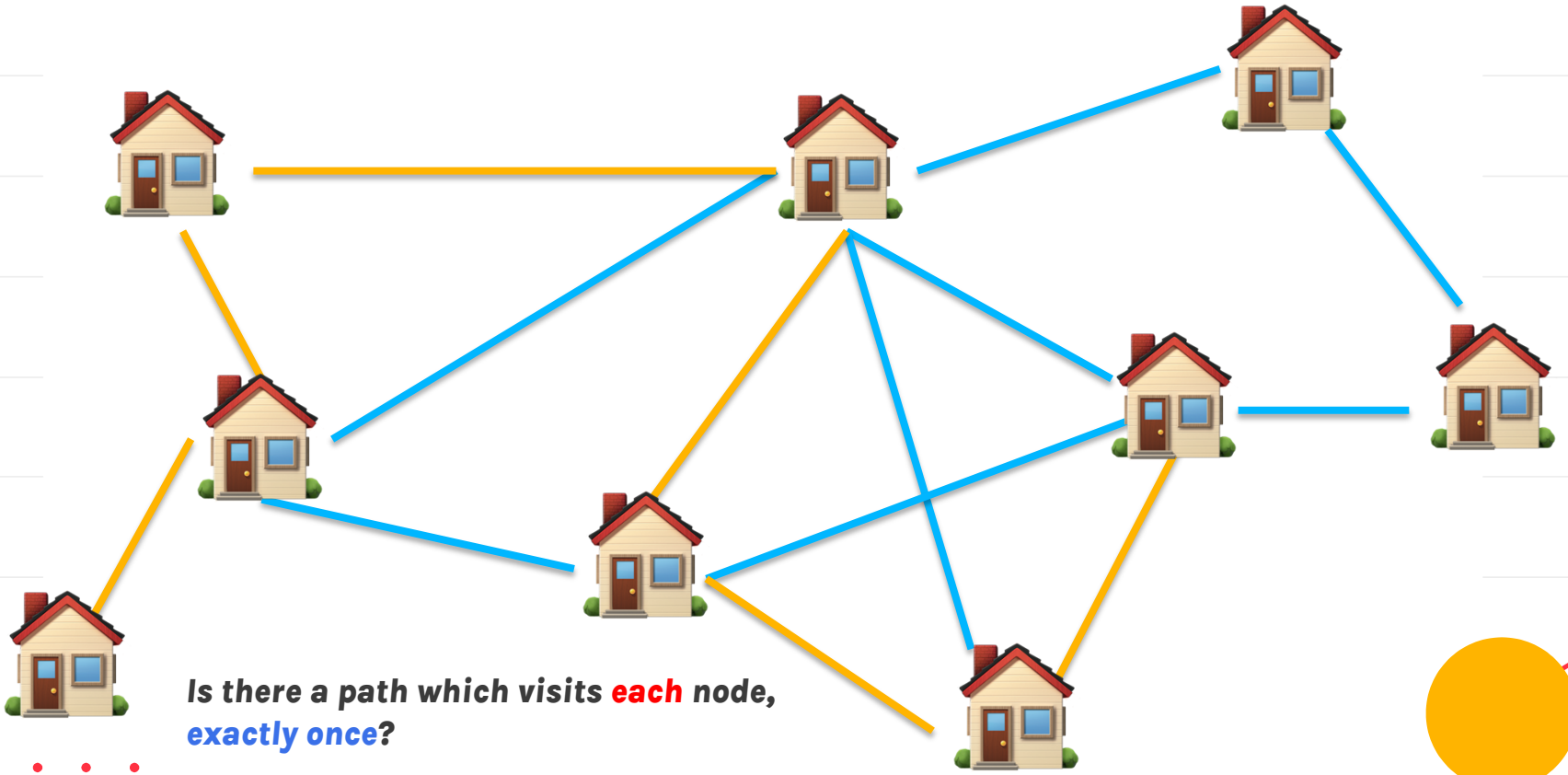
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



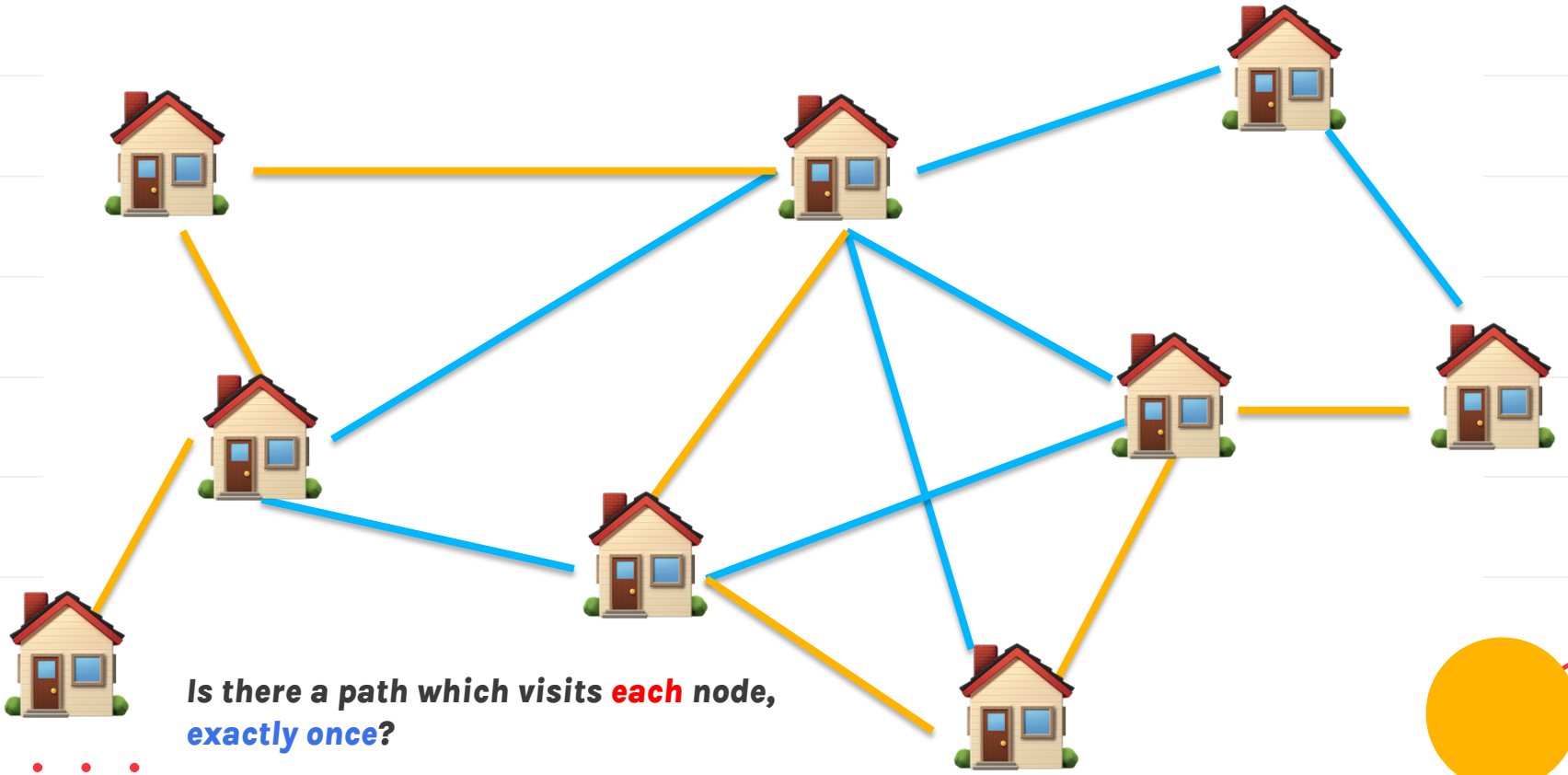
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



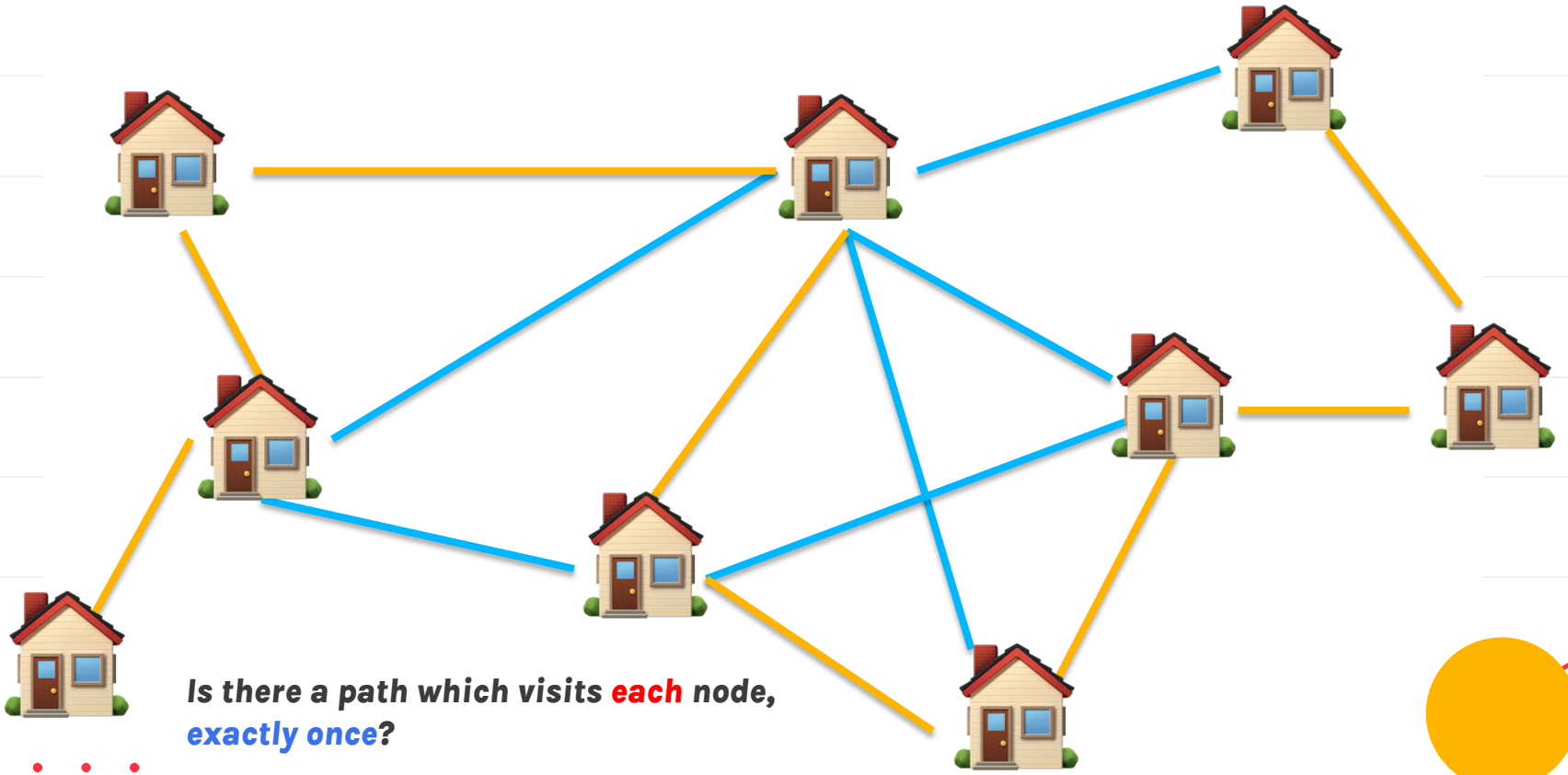
Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



Is there a path which visits each node, exactly once?

Practice: Hamiltonian Path



Is there a path which visits each node, exactly once?